

An Efficient Load Balancing Mechanism in Software Defined Networks

Kiran A Jadhav¹, Mohammed Moin Mulla¹, Narayan D. G¹

¹*School of Computer Science and Engineering*

KLE Technological University, Hubli, India

kiranjadhav226@gmail.com, moin.mulla@kletech.ac.in, narayan_dg@kletech.ac.in

Abstract - The growth of the network, increase in users, as well as surfacing of new technologies, such as cloud computing and big data has posed difficulties in managing traditional networks. To address this problem, Software Defined Network has been proposed which tends to govern the network in a better way. Traffic management technologies include traffic load balancing, QoS based scheduling. To fit in the prerequisites of quality of service, load balancing has to be considered as a crucial concept because it assists in controlling and regulating data traffic among numerous resources amplifying the network responsiveness, steadfastness and capacity. Network today needs to manage heavy traffic and serve clients in huge numbers at once. Using multiple servers with a load balancer is one of the solutions to this problem such that the clients can direct the request to load balancer which can further forward these requests to distinct servers after being subjected to load balancing strategy. Load balancing in a conventional network is based on the native information of the network and is non-programmable. But, SDN controllers have a global vision of the network and are also programmable which helps them design optimized load balancing mechanisms. In this paper, we have implemented Round Robin load balancing strategy and Random load balancing strategy using an OpenFlow switch. Mininet simulation results using round robin algorithm have led to higher throughput and decreased packet loss when compared to default random scheduling algorithm.

Keywords: SDN, OpenFlow, Mininet, POX controller, Load Balance, Random, Round-Robin.

I. INTRODUCTION

The Internet today connects every nook and corner of the world and has made everything reachable from anywhere. Despite their wide acceptance, traditional IP networks are complicated to configure and supervise because of their predefined policies. Reconfiguration of the network in response to flaws, defects, loads and fluctuations is tedious. Traditional networking uses integrated hardware and software to direct traffic across a series of routers and switches. These are incorporated perpendicularly binding both the control and data plane together. Creation of our own algorithms is not possible in

traditional load balancers since they are vendor specific and cannot be programmed.

SDN [1] focuses on the idea of partitioning the control plane that manages the network and the data plane where the traffic flows, granting the network administrators to chisel the traffic with a global centralized control. OpenFlow protocol is used to connect with the network devices which takes the control operations out of the individual controllers from the control plane and places them in a logical centralized controller. The controller typically handles configuration management of the SDN compliant devices, provides performance and fault management and also understands the network topology. Loaded with the above details, it can process connection requests based on desired requirements such as QoS and can perform link management between devices. SDN also introduces programmability, new abstractions in the network which facilitates network expansion. Virtualizing the network comes with advantages like the networks can be spun up and down dynamically and they can be fine-tuned for specific application use cases. Security policies can be installed on each individual network.

SDN [2] can be used to connect data centres to public cloud providers creating a hybrid cloud network with dynamic scaling ability. Breaking the barriers of layering, the control embraced by SDN not only includes packet forwarding at a switching level but also link tuning at a data link level. Based on both instantaneous network status and user defined policies, SDN permits a real-time centralized control of a network with an ability to acquire instantaneous network status. Specifically, SDN allows for a centralized control with a global network view and a feedback control with information exchanged between different layers in the network architecture. With properly designed, planned centralized algorithms many challenging performance problems can be quite effectively handled. It indicates that the new solutions for traditional problems such as load balanced packet routing, data traffic scheduling, Quality of Service (QoS) support, energy efficient operation and end-to-end congestion control can be developed and easily deployed to verify and validate their success in enhancing network performance [18]. Load

balancing is a computer networking method to distribute workload across multiple computers or a computer cluster, network links, central processing units, disk drives, or other resources. Load Balancing aims to achieve optimal resource utilization, maximize throughput, minimize response time, avoid overloading and avoid crashing. With Load Balancing physical servers from rotation can be removed transparently and easily so that maintenance can be performed on that server. It also helps in resource sharing by running multiple instances of an application/service on a server based on analyzed data. Static load balancing performs the task of mapping a static process graph onto a fixed hardware topology so that dilation can be minimized and load difference can be processed whereas Dynamic load balancing is highly desirable in a distributed system to evenly balance the load amongst the nodes reducing the mean job response time. Load Balancers [9] often referred to as application delivery controllers have evolved to provide capabilities like security, acceleration and authentication in a light weighted flexible software. Load Balancing in traditional networks is established based on the local information of the network which is not accurate or precise. Compared to the traditional network the main advantage of load balancing in SDN is that the forwarding decision calculations are centralized and not distributed, allowing multiple optional link utilization rates and flow characteristics to plan the strategy more comprehensively. Load balancing [16] in SDN makes the deployment of network services easier providing the necessary agility and cost structure. Through the use of controllers, SDN architecture with OpenFlow protocol offers better strategy for flows processing by network operators. At any moment a new server is added, the load balancing service comes in operation to take the appropriate action of dividing the data traffic among the servers at hand, considering load on the network and the computational capacity of individual server nodes respectively. Proposed work considers round robin and random algorithms for load balancing in Software Defined Networks. The paper discusses the performance of both the algorithms based on various parameters. We evaluate the performance of these algorithms using different scenarios in the Mininet emulator. The performance differentials are analyzed in terms of throughput and packet loss. We use a POX controller for programming the network switches. The remainder of this paper is organized as follows: After the introduction, the related works of SDN and load balancing are provided in Section II. Section III discusses the proposed methodology. Implementation of our proposed load balancing mechanisms is provided in Section IV. Section V discusses and analyzes the results, taking different scenarios into account. It also gives the comparison between the two methodologies. Section VI

discusses the conclusion of the paper with a view for future work.

II. Background Study

A. SDN Architecture

A SDN architecture is composed of networking elements such as routers, switches etc in the Data plane. These physical devices are nothing but simple forwarding elements without any software program to take independent decisions. The forwarding elements are instructed using flow rules on how to take actions for the received packets by southbound interfaces (eg, OpenFlow). The elements in the Data plane are managed by a control plane formed by control logic residing in applications and controllers. SDN controller offers an API representing the northbound interface to application developers. Generally to strategize forwarding devices, a northbound interface conceptualizes the low-level instructions worn by southbound interface. SDN with centralized validation also encourages evolution and innovation by providing an easy software implementation for new ideas/schemes, an adequate test environment with isolation and quick deployment using software updates.

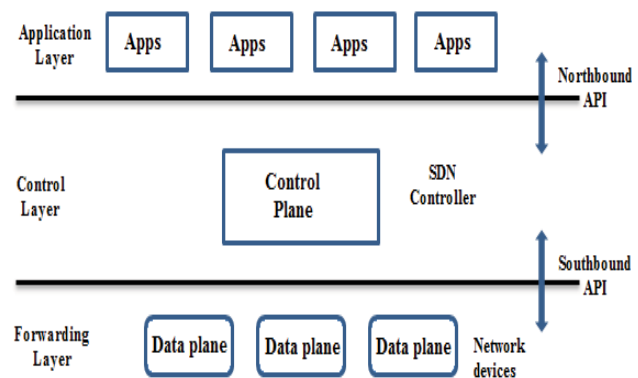


Fig 1: SDN Architecture

From Fig 1, we can see that SDN is a three-layer model ranging from an infrastructure layer to a control layer to an application layer, in a bottom-up manner. The separation of control plane and data plane can be recognized by an unambiguous and transparent programming interface between network devices (switches, routers) and the SDN controller. This decoupling between the two planes is pivotal to create abstractions in networking by disintegrating the network control issue into manageable segments thus contributing to network growth and revolution. Control plane manages network topology and makes decisions about where and how the traffic should be driven. It exercises complete control over the forwarding layer elements. The application layer consists of a set of applications that leverage the functionalities provided by the northbound interface to execute network authority. The

applications can be routing and load balancing as such. Here to generate the network configuration based on the network operator's rules and policies, a centralized logical controller (SDN controller) is the most important key bit.

B. Related Work

In [1], authors present an extensive survey on SDN by introducing the motive to study SDN, its key concepts and how it is different from traditional networking. They also describe the building blocks of SDN infrastructure using a bottom-up approach, southbound and northbound APIs, network operating system and its virtualization layers. In particular, the paper addresses the design of switches and control platforms concentrating on aspects of resilience, security, performance, dependability and scalability. In [2], authors review Load Balancing Strategies in SDN using deterministic and non-deterministic approaches. Considering load balancing parameters such as energy consumption, execution time, bit rate, latency and overhead, the paper provides the taxonomy and comparison of the discussed load balancing mechanisms. These represent the benefits of the chosen load balancing algorithms/techniques and investigate the metrics of the same. In [3], authors show a study of QoS aware routing in Software Defined hybrid networks. They schematize a hybrid network architecture of SDN using simulated annealing based QoS routing and learning bridge protocol which can recognize the existence of traditional switches by spanning tree protocol and thus have SDN hybrid network's global vision. The SAQR algorithm used in this paper can adjust the weights of delay, bandwidth requirement and loss rate to find the optimal path. The paper concludes with the presentation of an SDN hybrid environment where both SDN and legacy switches coexist. In [4], authors discuss Multipath Routing in SDN based Data Center Networks. They use multiple routing paths to calculate the number of paths used for a new incoming flow. Adaptive worst fit multipath routing algorithm can mitigate the congestion in the network and also can achieve a higher ratio of average packet delivery. It can distribute traffic to the least no of worst-fit paths and avoid sharing bottleneck links between paths, improving average end-to-end delay and throughput per flow. In [5], authors discuss the problem of load balancing arising due to allocation of cloud resources to users on demand. They have classified the Load balancing algorithms into Static and Dynamic algorithms. Considering the characteristics like fairness, throughput, performance, overhead and resource use and fault tolerance this paper mainly pinpoints on the concept of load balance and different measurement parameters which reduce energy consumption in a cloud computing environment. In [6], authors discuss the effective use of SDN in Traffic Engineering to reduce packet loss and delays which ultimately helps in significant improvement in network utilization. They have developed fully polynomial time approximation schemes to optimize the traffic engineering in SDN. In the paper, they have shown

how deploying a few strategically placed forwarding elements of SDN can improve the overall network performance. In [14], authors have compared round robin and weighted round robin load balancer techniques for their performance in SDN. The load balancer in the traditional IP-based network stabilizes the load and if a new flow enters, the load balancing router picks a server in accordance with the on-going network status as a target server of the new flow. Then the packets are transmitted to its target server through the path computed by using routing protocols. In [16], the scalability of the QoS-aware Network Operating System (QNOX) is analyzed through a set of experiments. The paper provides details on network resource discovery and route calculation for each forwarding element in a network by centralized OSPF, fault notification and fault restoration methods.

This survey motivated us to carry out the performance estimation and analysis of different load balancing strategies. The currently proposed work focuses on the practical implementation of the load balancing mechanisms.

III. PROPOSED WORK

The SDN controller takes all the data path control decisions without depending on algorithms which are described by an additional network. Having system information in prior is important in static methodology. Load balancing can be implemented in software or in physical equipment. SDN controllers can be clustered for fault tolerance and high availability. SDN switches that support the modification of packet headers can function as a simple cost effective load balancing device. In an SDN network, the centralized controller has to finish a series of real-time least loaded server selections in order to fulfil the needed load balancing. A Load balancer has control over the complete network application and the rule is programmed in it. It can run multiple instances of an application on a server, can balance the data load based on analysis of data which saves running time.

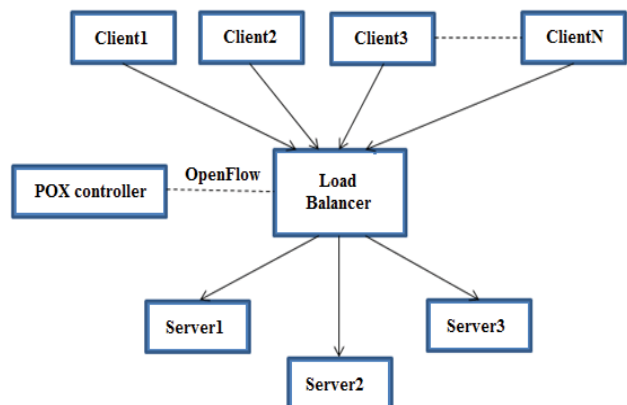


Fig 2: Load Balancing Architecture

Each server has its hardware/software limitation. Considering that each of the client hosts try to access about a hundred requests, a single server cannot handle such load. So, in a real-life scenario there is a pool of servers and a load balancer that is going to distribute the traffic across the servers. The load balancing architecture here in Fig 2 includes an OpenFlow switch with POX controller and multiple servers connected to the switch. Load balancer has a virtual IP which is exposed to the hosts so that they can communicate after which the traffic can be rerouted to the servers. When the client sends a request, the information in its packet header is compared with the flow entries in the OpenFlow switch. If the information matches then based on load balancing scheme, the switch adjusts the destination address to the address of one of the servers and directs the packet to that particular server. The switch forwards the packet to the POX controller if the information does not match with flow entry and the controller figures out how to handle the packet.

After collecting information about the two proposed algorithms Random and Round-Robin, we can discover the most efficient algorithm to distribute the load appropriately based on the traffic at each cluster.

Algorithm 1 shows the implementation for scheduling resource requests from client nodes and allocating servers to the client nodes using Random Selection Algorithm. In a given set of available servers, the Load Balancer arbitrarily chooses one server node for forwarding the request from client nodes.

Algorithm 1: Random Selection Algorithm

Input: Request, Set of available servers.
Output: Request allocation to servers.

```

Begin
  Flowtime = 20-80secs
  Get list of servers
  Whenever a new client request comes in
  Pick a random server from the list of servers using
  chosen_server = random.choice(SV_HOSTS)
  Assign the request to a random chosen server
End

```

In case the capacities of the servers are disproportionate, the load balancer running on Random algorithm might still distribute requests without considering it. This can result in over heading of a specific server which might affect the network.

Algorithm 2 shows the implementation for scheduling resource requests from client nodes and allocating servers to the client nodes using Round Robin Selection Algorithm. RR is the most widely used algorithm as it is easy to implement and understand. In here, each server node receives the requests from client nodes in a circular

pattern. Load Balancer allocates the request to several servers in a round robin manner. Also the specifications of servers such as CPU, RAM etc come into context so that the workload can be properly assigned to each of the servers. Thus, the round robin algorithm is best for clusters consisting of servers with identical specifications.

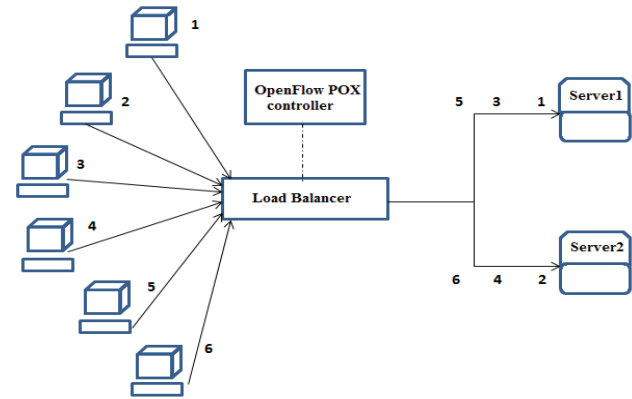


Fig 3: Round Robin Selection

Fig 3 describes the system model of Round Robin Selection strategy. A set of servers is designed and formulated to examine the load in a sequential order by rotation. Here, a single device is capable enough to process the same number of requests. Round Robin selection algorithm performs effectively in an analogous network environment. Round Robin algorithm is considered best for clusters consisting of servers with identical specs.

Algorithm 2: Round Robin Selection Algorithm

Input: Request, Set of available servers.
Output: Efficient request allocation to servers.

```

Begin
  Flowtime = 20-80secs
  Get list of servers
  Whenever a new client request comes in
  Pick a server from the list of servers using
  self.last_server_idx=(self.last_server_idx+1)% len(sv_hosts)
  chosen_server = sv_hosts[self.last_server_idx]
  Request is forwarded to each server in a cyclic fashion
End

```

IV. IMPLEMENTATION

Implementation of Random selection and Round Robin selection algorithm is done using a python based Pox controller with the help of a Mininet emulator. Mininet is a freely available, competent open source SDN emulator. Mininet builds a realistic virtual network and runs the kernel in a brief amount of time on a single machine. The machine could be native, virtual or cloud. It is easier to create, personalize and test SDN networks because Mininet

has a user friendly command line interface. Mininet also has a default controller. Any external controllers can be preferred to work in accordance to the network requirement.

Open Flow is a standard interface between the data and control plane of SDN which ensures communication. OpenFlow also embodies a direct pathway to the forwarding plane consisting of network devices like switches and routers. According to its flow table, OpenFlow forwards the packet from source to intended target. The flow table of this switch comprises a set of entries which consists of matching fields, a set of instructions to be followed for the dataset and counters. The POX controller of SDN supports OpenFlow and it embeds a control program that registers for events such as packet arrival or topology changes. It is relatively easy to read, write codes and is highly used in rapid prototyping and experimentation. Several topologies are considered for simulating the load balancing mechanisms in an OpenFlow based SDN environment with varying number of hosts using Mininet and a POX controller. Information of the topology is provided in the table below.

Table 1: Simulation Parameters

Parameters	Topology
Number of Controllers	1
Number of Switches	1-6
Number of Hosts	2-48
Controller	POX
Testing time	20-80 sec

V. RESULTS AND DISCUSSION

This section discusses the results obtained by simulating the proposed load balancing techniques in SDN using Mininet emulator. The performance of the proposed algorithms is compared using the QoS parameters average throughput and average packet loss which are defined as follows.

Average Throughput: It is the amount of data packets traveling through the communication channel successfully in a given period of time.

Average packet loss: It occurs when the data packets traveling through the communication channel fail to reach their target. Generally, packet loss by errors in transmission of data or network congestion or time-out error is high.

We have considered 3 different types of scenarios to analyze the performance of our proposed algorithms which are discussed as follows.

Scenario 1: Varying number of Server Hosts

In this scenario, we have taken a different number of servers 3, 6, 9 respectively and calculated the average throughput and packet loss. If there are many servers then the allocation of client requests is easier for the load balancing algorithms.

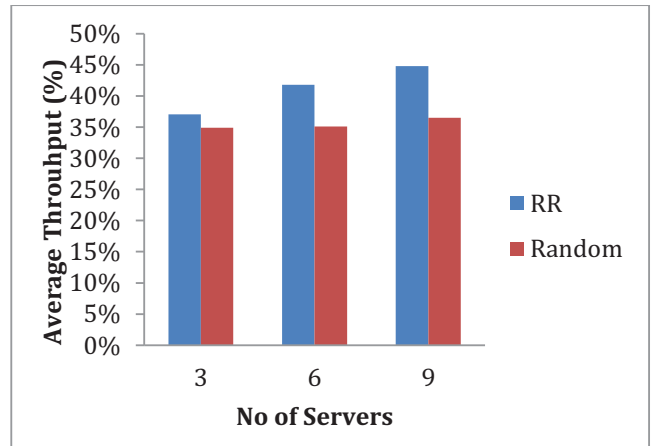


Fig 4: Throughput

From the graph in Fig 4 it is shown that there is a subsequent increase in the throughput rate of RR with the increase in number of server hosts. It is observed that RR selection in load balancing results in better throughput and lesser packet loss in comparison with Random Selection, when we increase the number of servers. The load on each of the routes is reduced and the response time is faster.



Fig 5: Packet loss

As shown in the graph of Fig 5 the rate in packet loss decreases with the increasing number of server hosts for Load Balancing with Round Robin selection algorithm. The packet loss for Random algorithm is more than RR because the packet flow during random selection does not

follow a particular way and the load calculation on a specific link can take quite some time.

Scenario 2: Varying number of Client Hosts

In this scenario, we have taken a varied number of clients 6, 12, 24, 48 respectively and tabulated the average throughput and packet loss in each case.

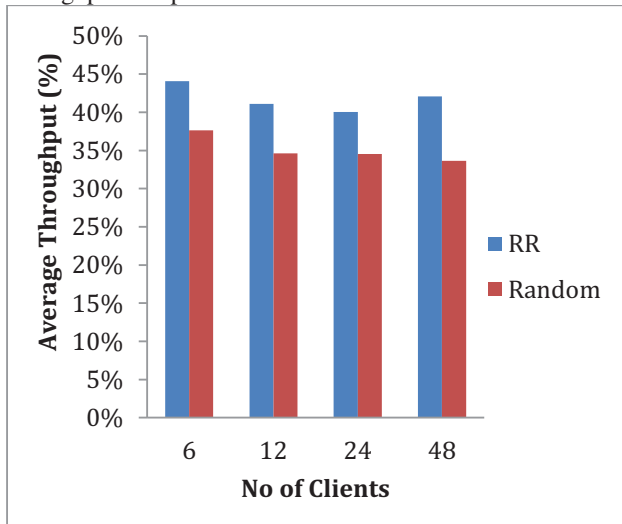


Fig 6: Throughput

The graph in Fig 6 depicts that the increased number of clients can lead to a more complex process involving overhead. As the number of client hosts increases, the rate of throughput for load balancing decreases. This in turn affects the performance of the network. But from both the graphs 5 and 6, it is clear that Round Robin Selection performs better when compared to Random Selection technique in load balancing with higher number of client hosts.

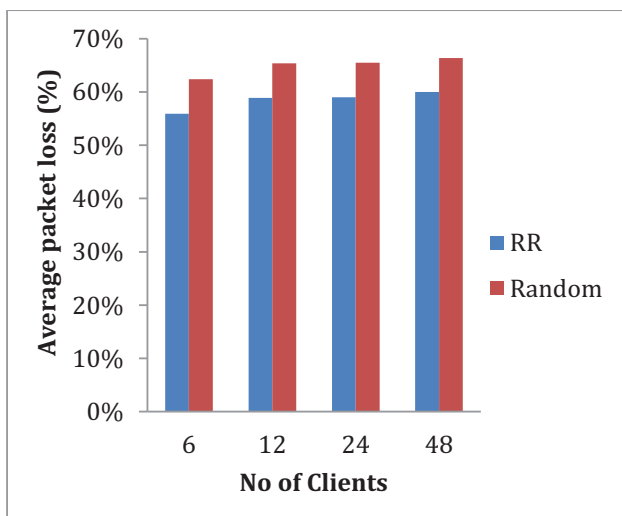


Fig 7: Packet loss

The graph in Fig 7 shows that the rate in packet loss increases with the number of clients for load balancing. This also may cause delay in the routing of packets. The overall packet loss for RR is less compared to random selection.

Scenario 3: Varying time

In this scenario, the time is considered keeping the no of client and server hosts constant. We have accounted the time in seconds running the topology for a flow timeout of 20, 40, 60, 80 sec respectively.

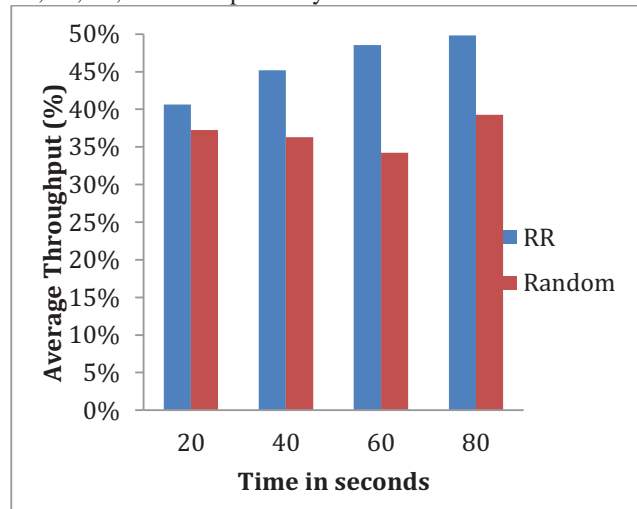


Fig 8: Throughput

The graph in Fig 8 shows that the throughput better with an increasing flow time. It is evident that the added runtime provides the load balancing selection algorithm to improve its performance, thus serving the client requests with lesser delay. As Round Robin selection serves the requests in an orderly fashion, it adds to the smooth running of the topology than Random selection.

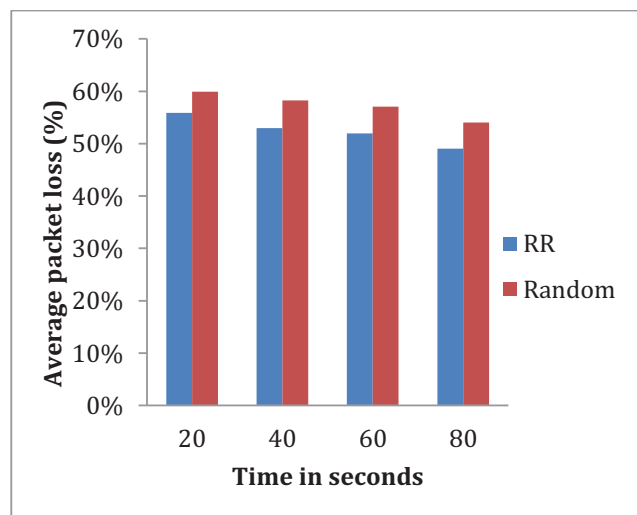


Fig 9: Packet loss

The graph in Fig 9 shows that the packet loss decreases with an increased flow time.

Higher throughput and lower packet loss elevates the network operation. When we look closely into each of the scenarios, we understand that Round Robin load balancing technique proves to be more efficient as it simply distributes the client requests across a set of servers turn by turn, considering the load on the each of the routes whereas Random load balancing allocates the client requests to the servers randomly.

VI. CONCLUSION AND FUTURE WORK

The difficulties encountered in conventional conservative network management are more and thus SDN is an evolving architecture model which solves these issues. SDN is an open and flexible paradigm that helps in the regulation of the network switches on a very large-scale network. Centralization of the forwarding information base allows optimum routes to be calculated deterministically for each flow end-to-end across the topology. SDN has gained tremendous momentum and as it is advancing towards an era of modern data networks, it requires a highly extensible and intelligent Load Balance Management. This paper outlines the design of SDN and compares it with conventional networks. And it is evident from the results that applying the concept of Round Robin selection for load optimization reduces the packet loss, increases the throughput and provides better performance in comparison to the Random Selection technique. Random Selection has large overheads. Round Robin Selection also avoids overloading of any single resource which minimizes resource consumption, congestion, leading to an effective load balance which enhances the network performance. This way SDN can dynamically respond to application requirements and optimize the utilization of the network without overlooking service quality.

As a future work, we plan to inculcate machine learning techniques in SDN to estimate the load on the servers and perform load balancing. SDN application's performance is also dependent upon the performance of the controller. So, we plan to use other controllers as well.

References

- [1] Kreutz Diego, Fernando M. V. Ramos, Paulo Esteves Verissimo, Christian Esteve Rothenberg, Siamak Azodolmolky, Steve Uhlig, "Software-defined networking: A comprehensive survey." *Proceedings of the IEEE* | Vol. 103, No. 1, January 2015
- [2] Wenfeng Xia, Yonggang Wen, Chuan Heng Foh, Dusit Niyato and Haiyong Xie, "A Survey on Software-Defined Networking" *IEEE communication surveys*, vol. 17, no. 1, first quarter 2015
- [3] Ali akbar neghab, Nima jafari navimipour, Mehdi hosseinzadeh and Ali rezaee1 "Load Balancing Mechanisms in the Software Defined Networks: A Systematic and Comprehensive Review of the Literature" *IEEE*, 2018
- [4] Chienhung Lin, Kuochen Wang, Guocin Deng "A QoS-aware routing in SDN hybrid networks" 12th International Conference on Future Networks and Communications (FNC 2017)
- [5] Yi-Chih Lei, Kuochen Wang and Yi-Huai Hsu "Multipath Routing in SDN-based Data Center Networks" 2015 European Conference on Networks and Communications (EuCNC)
- [6] Sajjan R.S, Biradar Rekha Yashwantrao "Load Balancing and its Algorithms in Cloud Computing: A Survey" (2017)
- [7] Sugam Agarwal, Murali Kodialam, T. V. Lakshman "Traffic Engineering in Software Defined Networks" 2013 Proceedings IEEE INFOCOM
- [8] Zhang, Hailong, and Xiao Guo. "SDN-based load balancing strategy for the server cluster." *Cloud Computing and Intelligence Systems (CCIS)*, 3rd International Conference IEEE 2014
- [9] Zhaogang shu, Jiafu wan, Jiaxiang lin, Shiyong wang, Di li, Seungmin rho, and Changcai yang "Traffic Engineering in Software-Defined Networking: Measurement and Management" *IEEE*, 2016.
- [10] Davoli, Luca, et al. "Traffic engineering with segment routing: SDN-based architectural design and open source implementation" *Software Defined Networks (EWSN)*, 2015 Fourth European Workshop on IEEE, 2015
- [11] Long, Hui, et al. "LABERIO: Dynamic load-balanced routing in OpenFlow-enabled networks." *Advanced Information Networking and Applications (AINA)*, 2013 IEEE 27th International Conference on. *IEEE*, 2013.
- [12] Namal, Suneth, et al. "SDN based inter-technology load balancing leveraged by flow admission control." *Future Networks and Services (SDN4FNS)*, 2013 IEEE SDN for. *IEEE*, 2013.
- [13] Li, Jun, et al. "An effective path load balancing mechanism based on SDN." *Trust, Security, and Privacy in Computing and Communications (TrustCom)*, 2014 IEEE 13th International Conference on. *IEEE*, 2014.
- [14] Qilin, Mao, and ShenWeikang. "A load balancing method based on SDN." *Measuring Technology and Mechatronics Automation (ICMTMA)*, 2015 Seventh International Conference on. *IEEE*, 2015.
- [15] Anam Ikram, Subhan Arif, Nafees Ayub, Waqas Arif, "Load Balancing In Software Defined Networking (SDN)" *MAGNT Research Report*, Vol.5(1). PP. 298-305, 2018
- [16] Sukhveer Kaur , Japinder Singh , Krishan Kumar , Navtej Singh Ghumman , "Round-Robin Based Load Balancing in Software Defined Networking" 2015 2nd International Conference on Computing for Sustainable Global Development (INDIACom)
- [17] Kwangtae Jeong, Jinwook Kim and Young-Tak Kim, "QoS-aware Network Operating System for Software Defined Networking with Generalized OpenFlows" 2012 IEEE/IFIP 4th Workshop on Management of the Future Internet (ManFI)
- [18] Mulla, Mohammed Moin, M. M. Raikar, M. K. Meghana, Nagashree S. Shetti, and R. K. Madhu. "Load Balancing for Software-Defined Networks." In *Emerging Research in Electronics, Computer Science and Technology*, pp. 235-244. Springer, Singapore, 2019.